

مستندات جامع طراحی فنی و معماری زیرساخت

سیستم ایجنتیک هوشمند رصد و کشف بلادرنگ تقاضای خرید در بسترهای پیام‌رسان



main @ 16d51c3	نسخه پایدار مخزن:	سامانه پیدا (مشتری‌یاب هوشمند)	نام پروژه:
Gemini 2.5 Flash Lite, Gemini 3.5, bge-m3	مدل‌های زبانی:	Agentic AI, Information Extraction, Hybrid Retrieval	دسته‌بندی و حوزه:
Telethon MTPROTO, Django 5.x, Gunicorn, Xray	کلاینت و وب‌سرویس:	Multi-Container Docker, PostgreSQL 16 + pgvector	معماری و دیتابیس:
پاییز ۱۴۰۳ (اکتبر ۲۰۲۶)	تاریخ انتشار سند:	Pay-as-you-go Token Ledger + Fair Sharing	مدل مالی مصرفی:

این سند فنی به صورت اختصاصی جهت ارزیابی در رویداد کوئرا تهیه شده و به طور دقیق منعکس‌کننده پیاده‌سازی واقعی کدهای مخزن گیت‌هاب است.

۱. بیان مسئله و چالش‌های مهندسی (Problem Formulation)

تجارت اجتماعی (Social Commerce) در ایران بستری غیرمتمرکز است که بیش از ۴۵ میلیون کاربر و صدها هزار فروشگاه فعال را میزبانی می‌کند. خریداران به جای جستجو در موتورهای سنتی، نیازمندی‌های خود را به صورت روزمره در سوپرگروه‌ها و کانال‌های تخصصی مطرح می‌کنند («بچه‌ها هندزفری زیر یک تومن چی خوبه؟»، «کسی دوره طراحی UI شیراز سراغ داره؟»). تحلیل خودکار این تقاضاها با سه چالش بنیادین مواجه است:

- زبان عامیانه و ساختار نامنظم:** خطاهای املایی، پیام‌های مقطع در چند خط، ارجاعات ضمنی به پیام‌های قبلی (Reply context)، و عدم وجود کلمات کلیدی مشخص.
- هزینه بالای استریمینگ مدل‌های زبانی:** ارسال تمام پیام‌های چت به LLM هزینه غیرقابل توجیهی دارد (صدها هزار پیام روزانه). نیاز به مکانیزم فیلترینگ رایگان بدون هوش مصنوعی احساس می‌شود.
- خطای توهم مدل در تطبیق کالا:** اگر نمره‌دهی به مدل واگذار شود، عدم رعایت سقف بودجه خریدار یا محدودیت مکانی شهر به علت توهم مدل رخ می‌دهد. سیستم نیازمند نمره‌دهی قطعی ریاضیاتی در کد است.

۲. معماری کلی سیستم و مایکروسرویس‌ها (System Architecture)

سامانه «پیدا» با الگوی مایکروسرویس بر بستر Docker طراحی شده و دارای ۶ کانتینر مستقل است که وظایف را بدون مسدودسازی یکدیگر انجام می‌دهند:

```
+-----+ | DATA INGESTION & NETWORK LAYER | | Telegram Supergroups / Channels (MTProto) X (Twitter Collector - Optional) | +-----+
+-----+ | v +-----+
+-----+ | [crawler] Telethon MTProto Client -> Rate Limiter / FloodWait -> crawler.tg_messages |
+-----+ | v +-----+
+-----+ | [db] PostgreSQL 16 + pgvector | | Schema: "crawler"
(Raw msgs) | Schema: "need_engine" (Vectors/Needs) | Schema: "public" | +-----+
+-----+ | (Read-Only) ^ (Publish Opportunities) v | +-----+
+-----+ | [engine] Autonomous Need Engine | | Pre-filter -> Access
Policy -> Need Extraction -> Hybrid Retrieval -> Deterministic | | Scoring -> Personalized Reply ->
need_engine.opportunities & need_engine.costs | +-----+
+-----+ | v +-----+
+-----+ | [sync] Synchronization Daemon & Billing Engine | | Reads seq cursor -> Upserts public.Opportunity / Customer -> Deducts
Seller Wallet | +-----+
+-----+ | v +-----+
+-----+ | [web] Django 5.x / Gunicorn 23.x ->
WhiteNoise -> Ops Dashboard (/ops/) & Opportunities | | [xray] V2Ray/Xray Core -> SOCKS5 :1080 / HTTP :8080 (Bypassing
network blocks on Iran) | +-----+
```

سرویس داکر	دستور اجرا	مسئولیت اصلی	ایزولاسیون دیتابیس
crawler	telegram_crawler.main	آرشیو پیام‌های سوپرگروه‌ها و استخراج بیهوش فرستندگان	نوشتن در اسکیمای crawler
engine	need_engine run	استخراج نیاز، تطبیق برداری و تولید پیش‌نویس پاسخ	خواندن Read-Only؛ نوشتن در need_engine
sync	sync_opportunities -- follow	پل انتقال به پل و تسویه مالی توکن‌ها با کیف پول	نوشتن در اسکیمای public
web	gunicorn config.wsgi	داشبورد ادمین و اپراتوری، کاتالوگ و فید فرصت‌ها	مدیریت جداول public
xray	teddysun/xray	پروکسی امن داخلی جهت رفع فیلترینگ تلگرام و جمینای	فقط در سطح شبکه داخلی داکر

۳. مدل ذخیره‌سازی داده و تفکیک اسکیمایی (Data Modeling)

برای رعایت اصول مهندسی نرم‌افزار و تضمین عدم آلودگی داده‌های پایگاه داده، دیتابیس PostgreSQL 16 به ۳ اسکیمای مستقل با سطوح دسترسی تفکیک‌شده مجهز شده است:

۱. اسکیمای کراولر (crawler)

وظیفه نگهداری پیام‌های خام تلگرام با وفاداری کامل به ساختار MTPROTO:

- tg_messages**: شناسه چت، فرستنده، نام کاربری، بیوگرافی
- فرستنده (**sender_bio**)، متن خام پیام، و پیوند به پیام والد (**reply_to_text** , **reply_to_msg_id**) تا ۳ سطح کانتکست.
- tg_users** و **tg_chats**: متادیتای کانال‌ها و شناسه‌های عددی اعضا.

تضمین ایزولاسیون داده خام: پروسه need_engine با فلگ دستوری default_transaction_read_only به دیتابیس متصل می‌شود. تست‌های خودکار یکپارچگی پیش از اجرا، عدم تغییر در هش رکوردهای کراولر را تایید می‌کنند.

۲. اسکیمای موتور تحلیل (need_engine)

نگهداری استیت و حافظه بلندمدت ایجنت بدون وابستگی به فریم‌ورک وب:

- needs**: کارت‌های نیاز استخراج‌شده، سقف بودجه، شهر، فوریت، و وضعیت (**resolved** / **open**).
- product_vectors**: بردارهای امبدینگ کاتالوگ با نوع داده **vector(768)** در افزونه pgvector.
- opportunities**: صف ترتیبی فرصت‌های کشف‌شده با کلید افزایشی **seq** برای خواندن بدون قفل توسط سرویس همگام‌ساز.
- costs**: لجر مالی تفکیکی به ازای مدل، مرحله، و شناسه فروشنده (تسهیم هزینه توکن).
- cache**: کش کلید-مقدار پرامپت‌ها و پاسخ‌ها جهت صرفه‌جویی در هزینه.

۳. اسکیمای عمومی پنل مدیریت (public)

جداول Django برای عملیات تجاری و تعامل با فروشندگان:

نقش در چرخه سیستم	فیلدهای کلیدی	مدل (Django Model)
مدیریت جوامع تحت رصد. تغییرات هر ۳۰ ثانیه توسط کراولر لود می‌شود.	link, scope (GLOBAL/PRIVATE), is_active, total_messages	MonitoredCommunity
فرصت نهایی تاییدشده در پنل با نمره کیفی ۰ تا ۱۰۰.	opportunity_id, status, score, customer, business	Opportunity
تحلیل ساختاریافته استخراج‌شده از پیام خریدار به همراه شواهد متنی.	intent, budget_max, city, urgency, raw_need	AIAnalysis
مدیریت داینامیک سرویس‌دهنده‌های مدل در مسیر ./ops/providers/	base_url, api_key, input_price_usd, output_price_usd, roles	Provider & AIModel
کیف پول فروشنده و لجر کسر هزینه به ازای هر فراخوانی استخراج و پاسخ.	balance_toman, amount_toman, kind, cost_usd, calls	Wallet & Transaction

معماری برداری در PostgreSQL 16 (pgvector Extension)

```
CREATE TABLE need_engine.product_vectors ( product_id BIGINT PRIMARY KEY, title TEXT NOT NULL, embedding vector(768) NOT NULL ); CREATE INDEX ON need_engine.product_vectors USING hnsw (embedding vector_cosine_ops);
```

۴. پایپ‌لاین ۹ مرحله‌ای اجرای ایجنت (Agent Execution Pipeline)

چرخه حیات تحلیل هر پیام به صورت یک زنجیره دقیق، قابل ردیابی و مهندسی‌شده پیاده‌سازی شده است:

گام ۱: پیش‌فیلتر رایگان بدون هوش مصنوعی (Zero-Cost Pre-filter)

پیام‌ها قبل از تماس با مدل‌های زبانی فیلتر می‌شوند: استیکرها، ایموجی‌های تکی، پیام‌های سیستمی تلگرام (Join/Leave)، پیام‌های ربات‌ها (`is_bot=True`)، دستورات خطی (مانند `/start`) و پیام‌های کمتر از ۴ کلمه بدون هزینه حذف می‌شوند. این مرحله بیش از ۶۵٪ از بار ترافیکی و هزینه‌های دلاری را فیلتر می‌کند.

گام ۲: اعمال قانون دسترسی منبع (Source Access Policy)

بر اساس وضعیت منبع در جدول `MonitoredCommunity` :

- منبع عمومی (Global):** متعلق به پلتفرم است. محصولات کاتالوگ همه فروشندگان اجازه تطبیق دارند. هزینه مرحله استخراج اولیه بر عهده پلتفرم است.
- منبع اختصاصی (Private):** توسط یک فروشنده مشخص معرفی شده است. سیستم تضمین می‌کند که پیام‌ها فقط با کالاهای همان فروشنده مقایسه شوند. هزینه استخراج پیام‌ها بین مالکان تقسیم مساوی می‌شود.

گام ۳: پنجره‌بندی زمانی (Batching & Temporal Windowing)

به جای پردازش تک‌پیام (که مفهوم زمینه مکالمه را از بین می‌برد)، پیام‌ها در بسته‌های ۵۰ تایی (`NE_TRIGGER_COUNT=50`) به همراه ۱۰ پیام ماقبل به عنوان Context وارد مدل می‌شوند. وقفه‌های زمانی بین پیام‌ها با نشانگر `||` علامت‌گذاری می‌شود.

گام ۴: استخراج ساختاریافته کارت نیاز (Need Extraction)

فراخوانی مدل زبانی استخراج (مانند `gemini-2.5-flash-lite`) و دریافت ساختار داده معتبر Pydantic شامل:

```
class NeedCard(BaseModel):
    summary: str          # خلاصه نیازمندی (مثال: "هدفون بی‌سیم نوین کنسلیگ")
    intent: Intent         # Implicit یا صریح (Explicit)
    budget_max: float | None # سقف بودجه عددی به تومان (مثال: 1800000)
    city: str | None       # شهر اعلام‌شده در پیام یا بیوگرافی فرستنده
    urgency: Urgency       # فوری، عادی، تحقیق اولیه
    requirements: list[str] # لیست شروط فنی (مثال: ["عمر باتری بالا", "گارانتی اصلی"])
```

گام ۵: حافظه مخاطب و ردیابی وضعیت نیاز (User Memory & Lifecycle)

در صورت وجود رکورد باز از همان فرستنده (بر مبنای `user_ref`):

- اگر پیام مکمل بفرستد، مشخصات کارت نیاز ادغام و به‌روزرسانی می‌شود.
- اگر عباراتی نظیر «خریدم»، «حل شد» یا «تهیه کردم» ارسال کند، وضعیت فرصت به `resolved` تغییر می‌یابد تا فروشنده بیهوده به او پیام ندهد.

۵. الگوریتم بازیابی هیبرید و نمره‌دهی قطعی (Retrieval & Scoring)

اعمال فیلتر سخت بودجه (Hard Budget)

(Filter

Reciprocal Rank Fusion

بازیابی دوگانه معنایی و واژگانی (RRF)

اگر خریدار بودجه‌ای اعلام کرده باشد (مثلاً ۲ میلیون تومان)، محصولاتی که قیمت آنها بیش از ۳ برابر بودجه باشد، فوراً و بدون ارسال به مدل راستی‌آزمایی حذف می‌شوند تا از پیشنهاد کالاهای بی‌ربط و نجومی جلوگیری شود.

حذف قطعی خطای توهم مدل: مدل‌های زبانی در مقایسه اعداد بزرگ و تبدیل ریال/تومان دچار توهم مکرر می‌شوند. این فیلتر به صورت کد پایتونی ۱۰۰٪ قطعی اجرا می‌گردد.

$$RRF_Score(d) = w_vec / (k + rank_vec(d)) + w_bm25 / (k + rank_bm25(d))$$

- جستجوی برداری کسینوسی: مقایسه امپدینگ خلاصه نیاز با `need_engine.product_vectors` روی ایندکس HNSW دیتابیس `pgvector`.
- جستجوی واژگانی BM25: تطبیق کلمات کلیدی، برندها و مدل‌ها روی ستون‌های متنی محصول.
- آستانه تطبیقی (Adaptive Floor): حذف کاندیداهای با شباهت کمتر از ۰.۵۵ در `bge-m3` جمینای یا ۰.۴۰ در `bge-m3`.

راستی‌آزمایی و نمره‌دهی قطعی در کد پایتون (Deterministic Scoring)

مدل زبانی راستی‌آزمایی (`use_verify`) تنها موظف به بازگرداندن یک پاسخ بولی ساختاریافته است و حق تعیین نمره عددی را ندارد:

```
class CandidateVerification(BaseModel):
    solves: bool # آیا محصول واقعاً مشکل خریدار را حل می‌کند؟
    requirements_status: dict[str, str] # وضعیت شروط: "met" | "unmet" | "unknown"
```

سپس امتیاز نهایی ۰ تا ۱۰۰ در مازول `scoring.py` بر اساس تابع زیر به صورت ریاضی محاسبه می‌شود:

مؤلفه ارزیابی	وزن نمره	منطق ارزیابی قطعی در کد پایتون
انطباق ماهیت کالا	۴۰ امتیاز	اگر <code>solves == True</code> باشد ۴۰ امتیاز کامل، در غیر این صورت کل محصول رد صلاحیت می‌شود.
انطباق بودجه (Price Fit)	۲۵ امتیاز	قیمت کمتر از بودجه: ۲۵ نمره کامل. بین ۱ تا ۱.۵ برابر بودجه: ۱۵ نمره. بالاتر: کسر تدریجی نمره.
الزامات فنی (Requirements)	۲۵ امتیاز	محاسبه نسبت شروط با وضعیت <code>met</code> نسبت به کل شروط الزامی خریدار.
موقعیت جغرافیایی (City Match)	۱۰ امتیاز	تطابق شهر خریدار با انبار/فروشگاه فروشنده: ۱۰ امتیاز کامل.

حد آستانه پذیرش: محصولات با نمره کمتر از ۱۵ (`NE_MIN_SCORE=15`) منتشر نمی‌شوند.

۶. نگارش پاسخ شخصی‌سازی‌شده و پل همگام‌ساز (Reply & Sync)

تولید پیش‌نویس پاسخ بر اساس لحن سازمانی

برای جلوگیری از پاسخ‌های ربات‌گونه و تکراری، سیستم از استایل پیام سرویس `sync_opportunities` به صورت یک پروسه مستقل اختصاصی فروشنده (`MessageStyle`) استفاده می‌کند:

- **لحن رسمی:** مناسب سازمان‌ها و خدمات تخصصی B2B.
- **لحن صمیمی و دوستانه:** مناسب کالاهای مصرفی، پوشاک و اکسسوری.
- **لحن مشاوره‌ای و تخصصی:** تمرکز بر پاسخ به دغدغه فنی خریدار پیش از معرفی قیمت.

در انتهای پیش‌نویس، لینک یکتای ارجاع درج می‌گردد:

```
http://<base_url>/r/<product_id>/?ref=<opportunity_id>
```

پل همگام‌ساز غیرمستودکننده (Sync Daemon)

سرویس `sync_opportunities` به صورت یک پروسه مستقل (Daemon) اجرا شده و وظایف زیر را بر عهده دارد:

- خواندن رکوردهای جدید از `need_engine.opportunities` بر اساس مکان‌نمای عددی افزایشی (`seq`).
- ایجاد یا به‌روزرسانی رکوردهای `Opportunity`، `Customer` و `AIAnalysis` در اسکیمای `public` به صورت کاملاً `Idempotent`.
- اجرای خودکار تابع `charge_pending_usage` جهت تسویه مالی توکن‌های جدید با کیف پول فروشندگان بدون مسدود کردن فرآیند اصلی.

نمودار جریان داده از کشف پیام تا نمایش در پنل فروشنده

```
[Telegram Chat] -> [crawler] -> [crawler.tg_messages] | v (Read-Only) [need_engine] (Extraction -> Retrieval -> Scoring -> Draft) | v (Write) [need_engine.opportunities] + [need_engine.costs] | v (Cursor seq polling) [sync daemon] | +-----+
-----+-----+ | | v v [public.Opportunity] [charge_pending_usage] [public.Customer] (Deduct Seller Wallet) [public.AIAnalysis] | | v +-----> [Seller Live Opportunity Feed]
```

۷. ارائه‌دهندگان مدل، ابزارها و مهندسی پرامپت (Models & Tools)

سیستم پیدا از معماری چند ارائه‌دهنده (Multi-Provider Support) بهره می‌برد. تمامی تنظیمات مدل‌ها در دیتابیس ذخیره شده و از مسیر </ops/providers/> قابل مدیریت است:

مزیت فنی	تعرفه ورودی / خروجی	نقش در پایپ‌لاین	مدل (Model ID)	سرویس‌دهنده (Provider)
سرعت فوق‌العاده زیر ۱ ثانیه و پشتیبانی عالی از فارسی عامیانه	\$0.30 / \$0.075 per 1M	استخراج نیاز و کارت کالا (Extract)	gemini-2.5-flash-lite	Google Gemini
استدلال منطقی قوی‌تر برای انطباق شروط خریدار	\$0.40 / \$0.10 per 1M	راستی‌آزمایی و نگارش پاسخ (Verify/Reply)	gemini-3.5-flash-lite	Google Gemini
فهم عمیق مفاهیم تجاری و چندزبانه	per 1M \$0.02 tokens	تولید بردار ۷۶۸ بعدی معنایی	text-embedding-004	Google Gemini
پایداری بالا در زمان اختلالات کلود گوگل	رایگان / ارزان	امبدینگ جایگزین (Cloudflare Workers)	bge-m3	Cloudflare AI
سرویس‌دهی بدون تحریم روی زیرساخت‌های بومی	متغیر بر اساس نرخ	پشتیبان اضطراری (Fallback)	gpt-4o-mini / localized	OpenRouter / Avalai

مهندسی پرامپت و خروجی ساختاریافته JSON

پرامپت‌های مازول [prompts.py](#) با بهره‌گیری از مهندسی Few-Shot به زبان فارسی طراحی شده‌اند:

- مدل زبانی ملزم به برگرداندن خروجی در قالب معتبر JSON با کلیدهای از پیش تعریف‌شده اسکیماست.
- هرگونه توضیحات اضافی، تگ‌های Markdown یا عبارات نامرتب در پرامپت سیستم ممنوع شده است.
- در صورت بروز خطای Parse در خروجی مدل، سیستم تا ۳ بار به صورت خودکار درخواست را با اصلاح پرامپت تکرار می‌کند.

سیستم کشینگ محلی پاسخ‌ها (LLM Response Cache)

جدول [need_engine.cache](#) تمام هش‌های پرامپت و پاسخ‌های دریافت شده را ذخیره می‌کند. پیام‌های مشابه یا پرامپت‌های مربوط به کارت‌های کالای بدون تغییر از کش خوانده می‌شوند. فراخوانی‌های کش‌شده برای فروشنده به صورت رایگان لحاظ شده و هزینه صفر در لجر ثبت می‌شود ([charge_cached=False](#)).

۸. کلاینت تلگرام و مدیریت محدودیت‌های نرخ (MTPROTO & Rate Limits)

چرایی انتخاب MTPROTO نسبت به Bot API

ربات‌های رسمی تلگرام (Bot API) در اغلب سوپرگروه‌ها به پیام‌های کاربران دسترسی ندارند (Group Privacy Mode) و نیاز به دسترسی ادمین دارند. سامانه پیدا از کلاینت کاربرمحور (Userbot) مبتنی بر کتابخانه Telethon و پروتکل مستقیم MTPROTO استفاده می‌کند که به صورت یک عضو عادی در گروه‌ها حاضر شده و پیام‌ها را استریم می‌کند.

مدیریت پایداری شبکه و عبور از فیلترینگ با Xray

روی سرورهای مستقر در داخل کشور ایران، کانترینر **xray** با پروتکل‌های مدرن VLESS/VMess ترافیک را تونل می‌کند. کراولر و کانترینرهای پایتونی با متغیرهای **TG_PROXY=socks5://xray:1080** و **HTTPS_PROXY=http://xray:8080** به صورت کاملاً پایدار و بدون قطعی داده‌ها را تبادل می‌کنند.

پارامترهای مدیریت نرخ و ضد اسپم تلگرام در کد پروژه

متغیر پیکربندی	مقدار پیش‌فرض	توضیح فنی و هدف امنیتی
TG_JOIN_INTERVAL_SECONDS	۶۰ ثانیه	حداقل فاصله زمانی بین پیوستن به دو گروه جدید جهت جلوگیری از مسدودسازی اکانت.
TG_MAX_GROUPS	۱۰۰ گروه	سقف منطقی تعداد گروه‌های همزمان تحت رصد توسط یک اکانت کراولر.
FLOOD_SLEEP_THRESHOLD	۶۰ ثانیه	در صورت دریافت خطای FloodWait، اگر مدت انتظار کمتر از ۶۰ ثانیه باشد، کراولر به خواب می‌رود.
FLOOD_MAX_WAIT_SECONDS	۹۰۰ ثانیه	حداکثر سقف مجاز توقف در خطاهای طولانی سرور تلگرام قبل از لغو و بازتلاش.
FLOOD_MAX_RETRIES	۳ بار	تعداد تلاش مجدد برای یک درخواست قبل از ثبت خطا در جدول جوامع.
TG_PROFILE_DELAY_SECONDS	۳۰ ثانیه	تاخیر در استعلام بیوگرافی کاربران جدید (users.GetFullUser) جهت عدم ارسال درخواست رگباری.

۹. راهنمای گام به گام استقرار و عملیات (DevOps Guide)

این بخش راهنمای عملیاتی اجرای سامانه روی سرور پروداکشن لینوکس (Ubuntu 22.04) را شرح می‌دهد:

مشخصات و الزامات سرور پروداکشن: آدرس سرور: 194.60.230.97 (پورت 9011 SSH) | مسیر اختصاصی: /var/www/ebi852/ | پورت وب: 8001.
دایرکتوری /var/www/coffeeCRM/ و پورت ۸۰۰۰ به پروژه دیگری تعلق داشته و نباید لمس شوند.

گام ۱: دریافت آخرین نسخه کد از مخزن گیت‌هاب

```
cd /var/www/ebi852
git fetch origin main && git checkout main && git pull origin main
```

گام ۲: ایجاد و پیکربندی فایل متغیرهای محیطی (env.)

```
cat << 'EOF' > .env
POSTGRES_DB=customer_yab
POSTGRES_USER=postgres
POSTGRES_PASSWORD=SECURE_DB_PASSWORD_HERE
POSTGRES_HOST=db
POSTGRES_PORT=5432
DJANGO_SECRET_KEY=SECURE_DJANGO_KEY_HERE
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=*
WEB_PORT=8001
TG_API_ID=12345678
TG_API_HASH=abcdef0123456789abcdef0123456789
NE_LLM_API_KEY=YOUR_GEMINI_API_KEY
NE_EXTRACT_MODEL=gemini-2.5-flash-lite
NE_VERIFY_MODEL=gemini-2.5-flash-lite
NE_REPLY_MODEL=gemini-2.5-flash-lite
# Optional proxy for domestic servers:
# TG_PROXY=socks5://xray:1080
# HTTPS_PROXY=http://xray:8080
EOF
```

گام ۳: لاگین تعاملی کلاینت تلگرام (یکبار به جهت ذخیره نشست داکر)

```
docker compose run --rm crawler login
```

گام ۴: ساخت و اجرای پایدار کانتینرها در پس‌زمینه

```
docker compose --profile proxy up -d --build
```

گام ۵: ایجاد کاربر مدیر ارشد و بارگذاری داده‌های پایه

```
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py seed_products
docker compose exec web python manage.py seed_global_sources docker/global_sources.example.txt
```

داشبورد مدیریت عملیات و وضعیت سلامت سرویس‌ها در آدرس <http://194.60.230.97:8001/ops/> در دسترس است.

۱۰. مدل اقتصادی و تسهیم عادلانه هزینه‌ها (Cost Engineering)

سیستم دارای موتور صورت حساب Pay-as-you-go است که مصرف دلاری توکن‌ها را دقیقاً محاسبه و تسویه می‌کند:

فرمول رسمی کسر از کیف پول فروشندگان:

مبلغ کسرشده (تومان) = هزینه واقعی دلاری توکن‌ها × نرخ دلار در پیل × (۱ + سود پلتفرم / ۱۰۰)

نرخ دلار پیش فرض: ۱۰۰,۰۰۰ تومان | مارک آپ پیش فرض پلتفرم: ۳۰٪ تا ۷۰٪

- تسهیم گروه عمومی (Global): هزینه فاز استخراج اولیه برای همه مشترک است و پلتفرم آن را می‌پردازد؛ فقط هزینه تطبیق و پاسخ برای فروشنده‌ای که فرصت به او تعلق گرفته کسر می‌شود.
- تسهیم گروه اختصاصی (Private): هزینه استخراج پیام‌های چت به طور دقیق بین فروشندگانی که گروه را اضافه کرده‌اند تقسیم مساوی می‌شود (ستون **part** در لجر **costs**).
- کنترل سقف اعتبار (Balance Enforcement): در صورت اتمام موجودی کیف پول، فرآیند تحلیل برای محصولات آن فروشنده متوقف می‌گردد تا هیچ زیانی متوجه پلتفرم نشود.

۱۱. جمع‌بندی و معیارهای ارزیابی داوران کوئرا (Evaluation Highlights)

معیار ارزیابی	رویکرد پیاده‌سازی شده در سامانه پیدا
۱. پایداری و تاب‌آوری (Robustness)	تفکیک کامل مایکروسرویس‌ها، مدیریت خطاهای ۵۰۳ و ریت‌لیمیت‌ها با Exponential Backoff، و عدم کرش در قطعی شبکه.
۲. صفر بودن توهم (Zero Hallucination)	سلب اختیار نمره‌دهی از مدل زبانی و استفاده از الگوریتم قطعی پایتون بر پایه تطابق بودجه، شهر و الزامات متنی.
۳. بهینه‌سازی شدید هزینه (Cost Optimization)	فیلتر رایگان بدون هوش مصنوعی (حذف ۶۵٪ پیام‌ها)، کشینگ پرامپت‌ها و پنجره‌بندی ۵۰ تایی پیام‌ها.
۴. آمادگی تجاری در دنیای واقعی (Production-Ready)	استقرار روی سرور لینوکس واقعی با داکر، پروکسی Xray سازگار با شبکه ایران، و پیل عملیاتی کامل.

کد منبع و مستندات آنلاین در ریپازیتوری پروژه موجود و آماده ارزیابی است.